

Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Advanced Network

Lab Report: Authentication and Task
Management

Supervised by:

Mr. Mohamed Lahcen BENSAAD

Prepared by:

Smail MERSAD

Academic Year: 2025 / 2026

1. Simple Access Token Usage

Generation & Storage

During login , the server generates a **simple access token** using a 32-byte random value:

```
// auth.js
const token = crypto.randomBytes(32).toString('hex');
await db.run('INSERT INTO access_tokens (user_id, token) VALUES (?, ?)',
[user.id, token]);
return { type: 'simple', token };
```

The token is then:

- **Returned to the server-side route**
- **Stored inside the database table `access_tokens`**, linked to the user via `user_id`
- **Sent to the browser as a cookie.**

DB schema (from `db.js`) supports this:

```
CREATE TABLE IF NOT EXISTS access_tokens (

  id INTEGER PRIMARY KEY AUTOINCREMENT,

  user_id INTEGER,

  token TEXT,

  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,

  FOREIGN KEY(user_id) REFERENCES users(id)

);`);
```

Verification

When a protected page is accessed (**/tasks-page**, **/profile**, **/admin/tasks-page**), the middleware reads the cookie manually:

```
// auth.js

const cookieHeader = req.headers['cookie'] || "";

cookieHeader.split(';').forEach(pair => {

    const [key, value] = pair.trim().split('=');

    if (key === 'token') token = value;

});
```

Then it verifies the token:

```
// auth.js

const row = await db.get(

    'SELECT users.id, users.username, users.role

    FROM access_tokens

    JOIN users ON users.id = access_tokens.user_id

    WHERE token = ?',

    [token]

);
```

If valid → the user identity is restored into:

```
req.user = { id: row.id, username: row.username, role: row.role };
```

If invalid → **redirect to /login**.

2. Session Cookies & Security

Setting the Cookie

After successful login:

```
res.setHeader('Set-Cookie', `token=${result.token}; HttpOnly; Path=/  
Max-Age=3600`);
```

Header name:

Set-Cookie

Value format:

token=<tokenvalue>; HttpOnly; Path=/; Max-Age=3600

This header sends a cookie named **token** to the browser, and the browser stores it automatically.

The Max-Age=3600 attribute makes the cookie last **1 hour** instead of disappearing when the tab is closed.

HttpOnly Flag

I DID use HttpOnly.

- Prevents JavaScript (via XSS) from reading the cookie
- Protects the authentication token from attacker scripts
- Essential for preventing session hijacking

DevTools Evidence

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition ...	Cross Site	Priority
_xsrf		localhost	/	2025-12-...	59						Medium
ph_phc_4URIAm1uYfj07j8kWSe0J8lc8lqn...		localhost	/	2026-05-...	436			Lax			Medium
rl_anonymous_id		localhost	/	2026-05-...	127			Lax			Medium
rl_page_init_referrer		localhost	/	2026-04-...	85			Lax			Medium
rl_page_init_referring_domain		localhost	/	2026-04-...	91			Lax			Medium
rl_session		localhost	/	2026-05-...	192			Lax			Medium
rl_trait		localhost	/	2026-05-...	210			Lax			Medium
rl_user_id		localhost	/	2026-05-...	214			Lax			Medium
token		localhost	/	2025-12-...	69	✓					Medium
username-localhost-8888		localhost	/	2025-12-...	325	✓					Medium
username-localhost-8889		localhost	/	2025-12-...	341	✓					Medium
username-localhost-8890		localhost	/	2025-12-...	345	✓					Medium

3. Retrieving User Identity

From Token to User ID

Once the simple token is verified, the middleware loads the user with:

```
req.user = { id: row.id, username: row.username, role: row.role };
```

This allows all following routes to know exactly **which user is authenticated**.

SQL Join Example

The exact query used:

```
const row = await db.get(  
  
  'SELECT users.id, users.username, users.role  
  
  FROM access_tokens JOIN users ON users.id = access_tokens.user_id  
  
  WHERE token = ?',  
  
  [token]  
  
);
```

This ensures every request maps a **token** → **user_id** securely.

4. Creating Tasks (Ownership)

Setting the Owner

The **POST /api/tasks** route uses the authenticated identity:

```
await db.run('INSERT INTO tasks (title, owner_id) VALUES (?, ?)', [title, req.user.id]);
```

The frontend sends only the **task title**:

```
await fetch('/api/tasks', {
  method: 'POST',
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title })
});
```

(User ID never comes from the client).

Security Consideration

If the client could send `owner_id`, a malicious user could do:

```
{ "title": "Hacked", "owner_id": 1 }
```

→ Creating tasks under another account.

The code implementation prevents this vulnerability (**ID spoofing**).

5. Listing Tasks (Data Isolation)

Filtering by Owner

Users only see their own tasks:

```
const task = await db.get('SELECT * FROM tasks WHERE id = ?',  
[req.params.id]);
```

The client UI (**user_tasks.html**) displays only those tasks:

```
tasks.forEach(task => {  
  
    const div = document.createElement("div");  
  
    div.className = "task";  
  
    div.innerHTML = `  
  
    <span>${task.title}</span>  
  
    <button onclick="deleteTask(${task.id})">Delete</button>  
  
    `;  
  
    container.appendChild(div);  
  
});
```

SQL Query

```
SELECT * FROM tasks WHERE id = ?
```

This enforces **per-user data isolation**.

6. Protecting Pages (Access Control)

Middleware Logic

All protected routes use:

```
verifySimpleTokenCookie ( () => db)
```

The middleware **verifySimpleTokenCookie** protects all pages:

1. Extract token from cookies
2. If missing → redirect **/login**
3. Validate token against DB
4. If invalid → redirect **/login**
5. Else → attach user to **req.user** and continue

Code:

```
if (!token) return res.redirect('/login');  
  
if (!row) return res.redirect('/login');  
  
req.user = { id: row.id, username: row.username, role: row.role };  
  
next();
```

Additionally:

```
requireRole('admin')
```

prevents unauthorized users from accessing admin pages.

Implementation

Example from `index.js`:

```
app.get('/admin/tasks-page', verifySimpleTokenCookie(() => db),  
requireRole('admin'), (req, res) => {  
  
    res.sendFile('admin_tasks.html', { root: './' });  
  
});
```

This ensures:

- Without a token → cannot access protected pages
- With invalid token → redirected to login
- Admin-only pages require "**admin**" role